Lecture 05: Delegates And Events

Objectives:

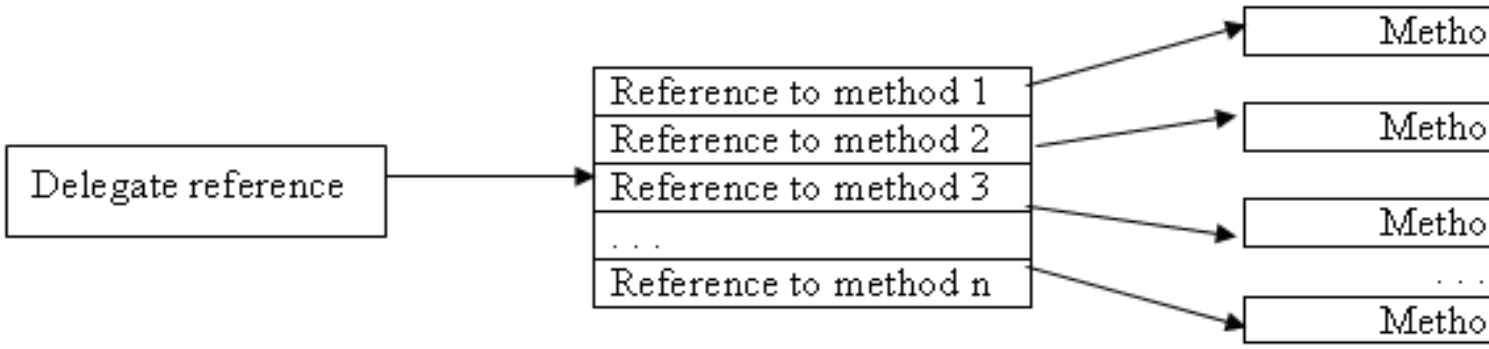
- Learn about delegates, how to create them and how to use them.
- Learn about special types of delegates called events.
- Learn how to use the standard event handler.

1. Delegates

A Delegate is a class whose declaration syntax is different from that of a normal class.

It is used to hold references to methods, so that when it is invoked, it automatically invokes all methods associated with it.

Thus, a delegate gives an indirect access to a method or methods, hence the name delegate.



To see how delegates are declared and used, consider the following example.

Example 1:

```
using System;

public class DelegateExample {
  public delegate void PrintingDelegate(String s);

  public static void Writer1(String s) {
    Console.WriteLine("From Writer1: "+s);
  }
```

```
  public static void Writer2(String s) {
    Console.WriteLine("From Writer2: "+s);
  }

  public static void Main() {
    PrintingDelegate d = new PrintingDelegate(Writer1);
    d("Hello There");

    //can point to more than one method
    d += new PrintingDelegate(Writer2);
    Console.WriteLine();
    d("Hello There");

    //can also point to instance method
    Console.WriteLine();
    MessageWriter mw = new MessageWriter();
    d+= new PrintingDelegate(mw.WriteMessage);
    d("Hello There");

    //You can also remove a method
    Console.WriteLine();
    d-= new PrintingDelegate(Writer1);
    d("Hello There");
  }
}

public class MessageWriter {
  public void WriteMessage(String s) {
    Console.WriteLine("From MessageWriter: "+s);
  }
}
```

```
From Writer1: Hello There

From Writer1: Hello There
From Writer2: Hello There

From Writer1: Hello There
From Writer2: Hello There
From MessageWriter: Hello There

From Writer2: Hello There
From MessageWriter: Hello There
Press any key to continue . . .
```

Delegate declaration

Notice that although delegate is a class, its declaration syntax is very similar to that of a method.

It is designed this way because a delegate can only hold references to specific types of methods – those methods whose signature matched that of the delegate.

Thus, in our example, the PrintingDelegate can only hold references to methods of the form:

[static] void MethodName(String s)

As the above example shows, such methods can be static or instance.

The above example declares the delegate as a nested class. This is usually how it is done, but in fact, a delegate can be declared on its own, like any other class.

Instantiating a Delegate

Before you create an instance of a delegate, you must have a method that you wish to associate that instance with. As we can see from the example, the syntax is:

```
DelegateType delegateVar = new DelegateType(methodName);
```

Notice that the method name must NOT be followed with parameters. Actually a method name without parameter means a reference to the method. So the above statement assigns the method reference to the ***delegateVar*** delegate instance.

Multicast Delegates

Internally, a delegate uses a linked list to hold references to the methods associated with it. Thus, a delegate instance can hold references to more than one method.

In fact internally, a delegate is declared as a subclass of the **System.MulticastDelegate** class, which in turn derives from the **System.Delegate** class. Thus, a delegate automatically inherits the methods of these classes. Check the methods inherited from these classes in the documentation.

Once a delegate instance has been created, we can assign more method references to it using the overloaded, += operator, or using the static ***Combine*** method of the **Delegate** class.

Similarly, a method reference can be removed from a delegate instance using the overloaded, -= operator or using the static **Remove** method of the **Delegate** class.

Invoking a Delegate
As we saw in the example, a delegate instance is invoked based on its method-like signature. In our example, since the associated methods are **void**, we simply call it as:

d(string)

This will automatically call methods associated with the delegate.

2. Events

A common application of delegates is in GUI programming where they are used as *call-back methods.*

That is, a class representing a GUI control such as the Button class will declare a public field for a delegate which it will invoke when the button is clicked.

An application that wishes to be notified when the button is clicked will write a method that should be executed when the button is clicked, and use the method to register with the delegate field.

When the user clicks the button, the method is automatically executed.

This procedure will work fine except for one problem – the delegate field is public.

This means other classes (other than the Button class) can invoke the delegate - thus raising a false alarm. In fact, the field can be assigned a new instance by another class – thus canceling any registration to the delegate made by other classes.

To solve this problem, C# introduced the **event** modifier.

If a delegate field is declared with the event modifier, then only the class in which it is declared can invoke it.

Also, the only operators allowed on event delegates (simply called **events**) outside their class of declaration are: **+=** and **-=**. Thus, other classes can only register themselves with the events on cancel their earlier registration.

Example 2: The following shows an example of using event-qualified delegates.

```
using System;

public delegate void EngineMonitor(String s);

public class Car {
  private int currentSpeed = 0;
  private bool isDead = false;
  private int maxSpeed;
  private String name;

  public event EngineMonitor Exploded = null;
  public event EngineMonitor AboutToExplod = null;


  public Car(String name, int maxSpeed){
    this.name = name;
    this.maxSpeed = maxSpeed;
  }

  public void Accelerate(int increment) {
    if (isDead) {
      if (Exploded != null)
        Exploded("The car has exploded");
    }
    else {
      currentSpeed += increment;
      if (currentSpeed >= maxSpeed) {
        isDead = true;
        if (Exploded != null)
          Exploded("The car has exploded");
      }
      else if (currentSpeed + 20 >= maxSpeed &&
AboutToExplod != null)
        AboutToExplod("Dangerous Speed:"+currentSpeed+", Car
```

```
about to explod");
        else
          Console.WriteLine("Current Speed = "+currentSpeed);

      }
    }
}

public class EventExample {
  public static void Main() {
    Car myCar = new Car("Corola", 200);

    //register with event source
    myCar.Exploded += new EngineMonitor(OnExplod);
    myCar.AboutToExplod += new
EngineMonitor(OnAboutToExplod);

    //speed up
    for (int i=0; i<10; i++)
      myCar.Accelerate(20);

    //cancel registration to events
    myCar.Exploded -= new EngineMonitor(OnExplod);
    myCar.AboutToExplod -= new
EngineMonitor(OnAboutToExplod);

    //no response
    for (int i=0; i<10; i++)
      myCar.Accelerate(20);
  }

  public static void OnExplod(String s) {
    Console.WriteLine("Message from car: "+s);
  }

  public static void OnAboutToExplod(String s) {
    Console.WriteLine("Message from car: "+s);
  }
}
```

```
Current Speed = 20
Current Speed = 40
Current Speed = 60
Current Speed = 80
Current Speed = 100
Current Speed = 120
Current Speed = 140
Current Speed = 160
Message from car: Dangerous Speed:180, Car about to explod
Message from car: The car has exploded
Press any key to continue . . .
```

3. The Standard Event Handler

Because the use of delegates to handle events is very common in GUI programming, C# has defined a special delegate named, **EventHandler**, which is used by most control classes to handle events.

The signature for such delegate is :

void (object source, EventArgs e)

where ***source*** is the object that fired the event and ***e*** contains any additional information about the event.

However, the EventArgs class is a class that does not have any fields that can be used to pass the event information to the client.

If there is a need to send event information, the programmer is expected to create a sub class from EventArgs, in which the desired fields and methods can be defined.

Example 3: The following example modifies the Car example to use the standard Event Handler class.

```
using System;

public class EventMessage : EventArgs {
    private string message;
  public EventMessage(string msg) {
    message = msg;
  }

  public string Message {
    get {return message;}
  }
}
```

```
public class Car {
  private int currentSpeed = 0;
  private bool isDead = false;
  private int maxSpeed;
  private String name;

  public event EventHandler Exploded = null;
  public event EventHandler AboutToExplod = null;

  public Car(String name, int maxSpeed){
    this.name = name;
    this.maxSpeed = maxSpeed;
  }

  public void Accelerate(int increment) {
    if (isDead) {
      if (Exploded != null)
        Exploded(this, new EventMessage("The car has
exploded"));
    }
    else {
      currentSpeed += increment;
      if (currentSpeed >= maxSpeed) {
        isDead = true;
        if (Exploded != null)
          Exploded(this, new EventMessage("The car has
exploded"));
      }
      else if (currentSpeed + 20 >= maxSpeed && AboutToExplod
!= null)
        AboutToExplod(this, new EventMessage("Dangerous
Speed:"+
                        currentSpeed+", Car about to explod"));
      else
        Console.WriteLine("Current Speed = "+currentSpeed);

    }
  }
}

public class EventExample {
  public static void Main() {
    Car myCar = new Car("Corola", 200);
```

```
    //register with event source
    myCar.Exploded += new EventHandler(OnExplod);
    myCar.AboutToExplod += new EventHandler(OnAboutToExplod);

    //speed up
    for (int i=0; i<10; i++)
     myCar.Accelerate(20);

    //cancel registration to events
    myCar.Exploded -= new EventHandler(OnExplod);
    myCar.AboutToExplod -= new EventHandler(OnAboutToExplod);

    //no response
    for (int i=0; i<10; i++)
     myCar.Accelerate(20);
  }

  public static void OnExplod(Object source, EventArgs e) {
    Console.WriteLine("Message from car:
"+((EventMessage)e).Message);
  }

  public static void OnAboutToExplod(Object source, EventArgs
e) {
    Console.WriteLine("Message from car:
"+((EventMessage)e).Message);
  }
}
```